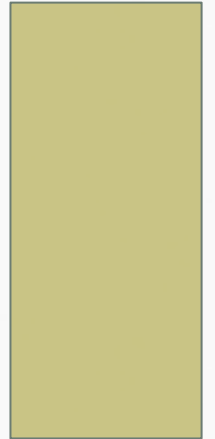


EC-211 DATA STRUCTURES

LECTURE 5
WALLS & MIRRORS: CHAPTER 4



Can you crack the code?



CODE

Will you crack the code?

5 4 8

One number is correct and well placed

5 3 0

Nothing is correct

1 5 7

Two numbers are correct but wrong places

8 0 6

One number is correct but wrong place

6 4 7

One number is correct but wrong place

DELETING A SPECIFIED NODE (N) FROM A LINKED LIST

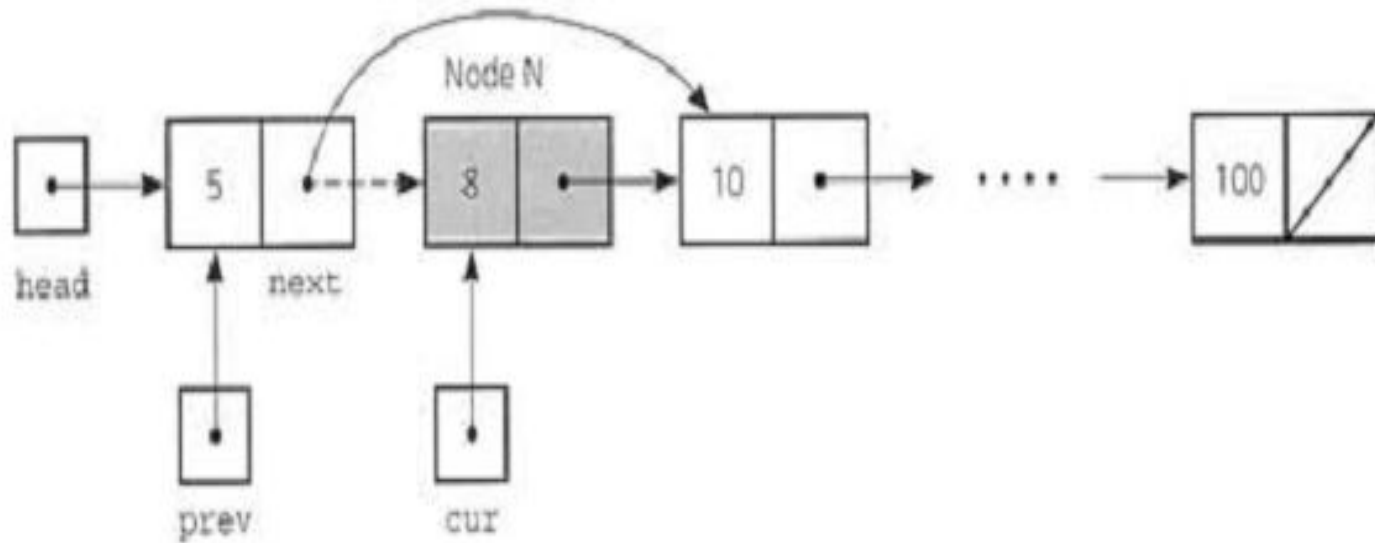


FIGURE 4-10

Deleting a node from a linked list

`prev->next=cur->next;`

DELETING A SPECIFIED NODE N FROM A LINKED LIST

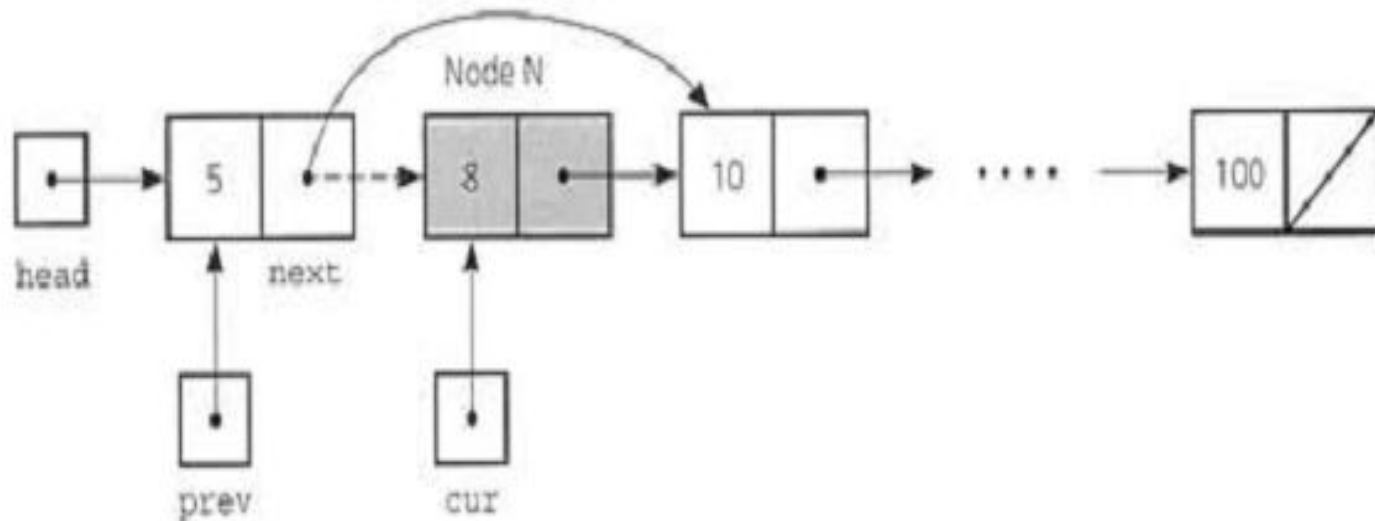


FIGURE 4-10

Deleting a node from a linked list

`prev->next=cur->next;`

Does this technique work for any node N in the list, regardless of N's position in the list?

DELETING A SPECIFIED NODE N FROM A LINKED LIST

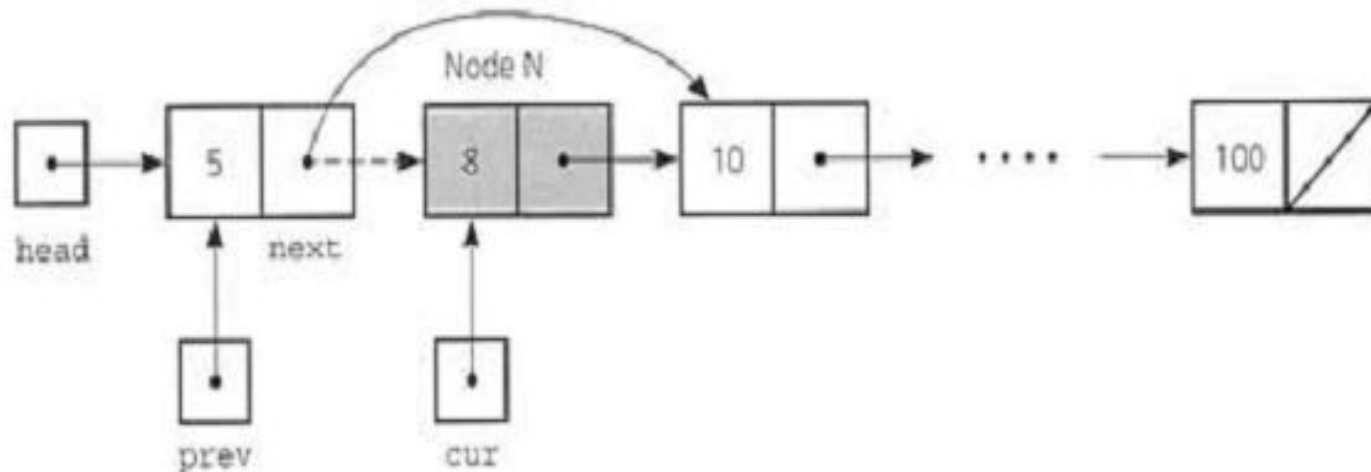


FIGURE 4-10

Deleting a node from a linked list

`prev->next=cur->next;`

Does this technique work for any node N in the list, regardless of N's position in the list?

No, It does not work if N is first node in the list

DELETING THE FIRST NODE IS A SPECIAL CASE

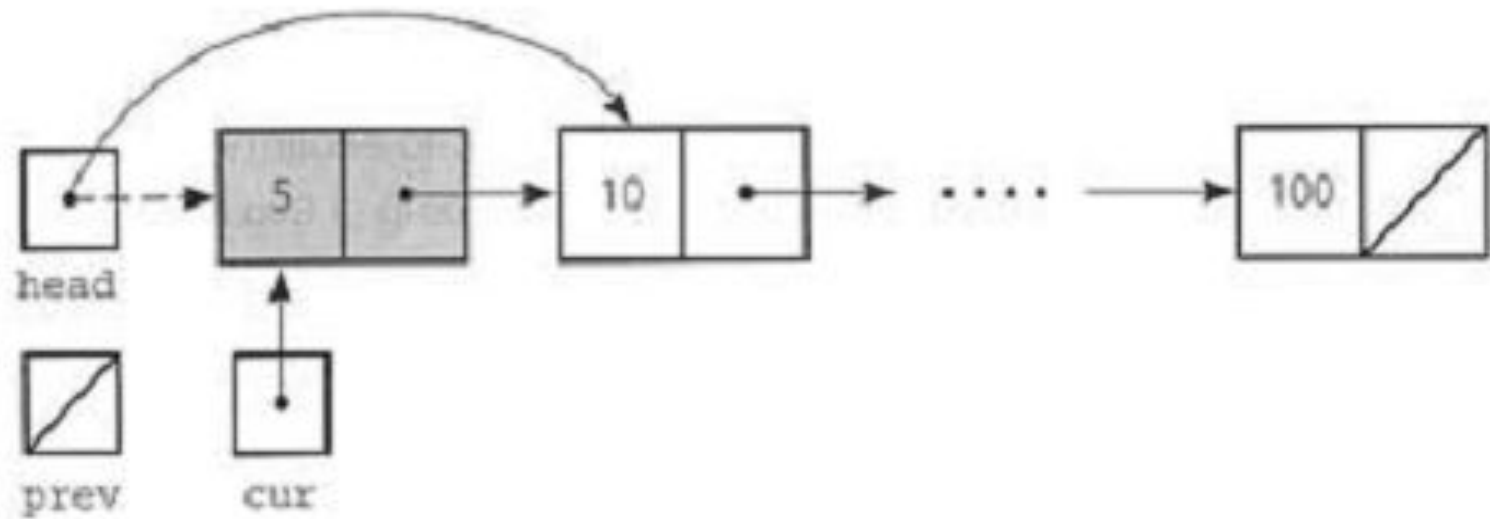


FIGURE 4-11

Deleting the first node

```
head=head->next;
```

RETURNING DELETED NODES TO THE SYSTEM

- **Before** you change value of current, you should use the statements:

```
cur->next=NULL;
```

```
delete cur;
```

```
cur=NULL;
```

RETURNING DELETED NODES TO THE SYSTEM

- **Before** you change value of current, you should use the statements:

```
cur->next=NULL;
```

```
delete cur;
```

```
cur=NULL;
```


HOW DO THE VARIABLES PREV & CUR COME TO POINT TO THE APPROPRIATE NODES?

- Consider the context of deletion, e.g.
 - Deleting by position
 - Deleting by data value
 - Etc.

SUMMARIZING DELETION

- Three high level steps:
 1. Locate the node that you want to delete
 2. Disconnect this node from the linked list by changing pointers
 3. Return the node to the system

INSERTING A NODE INTO A SPECIFIED POSITION OF THE LINKED LIST

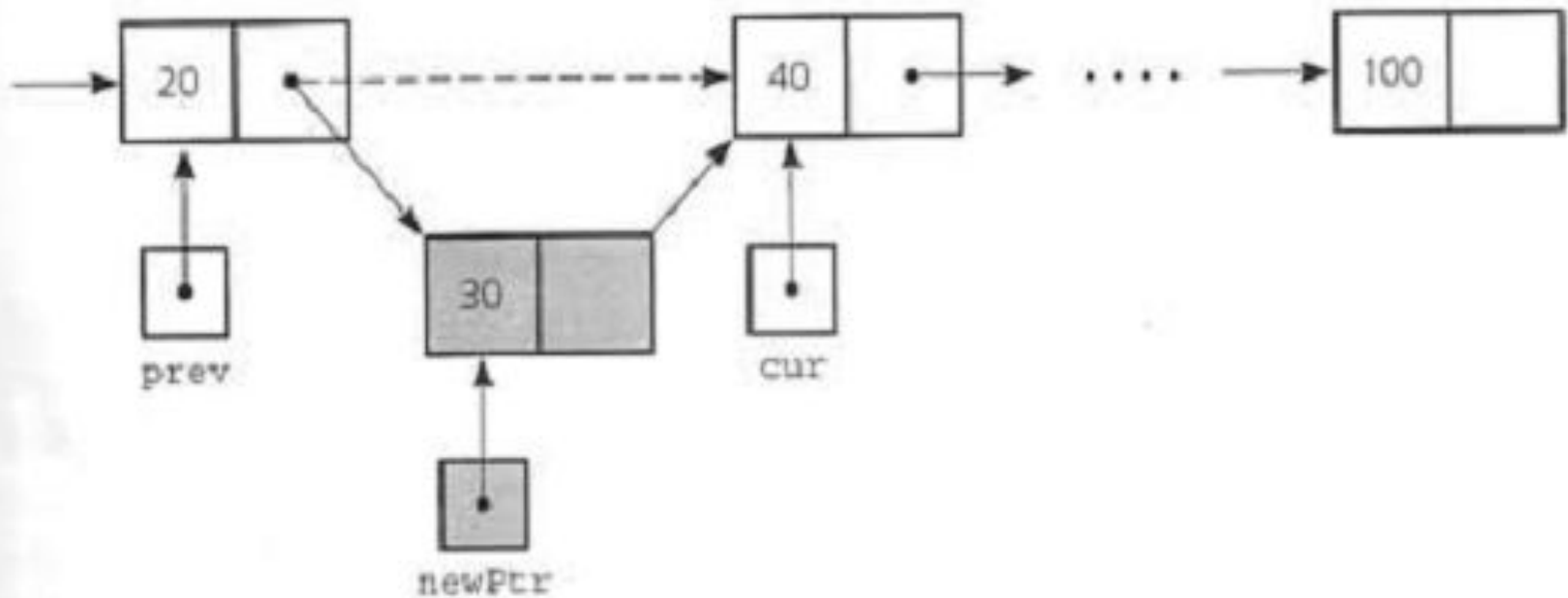


FIGURE 4-12

Inserting a new node into a linked list

INSERTING A NODE INTO A SPECIFIED POSITION OF THE LINKED LIST

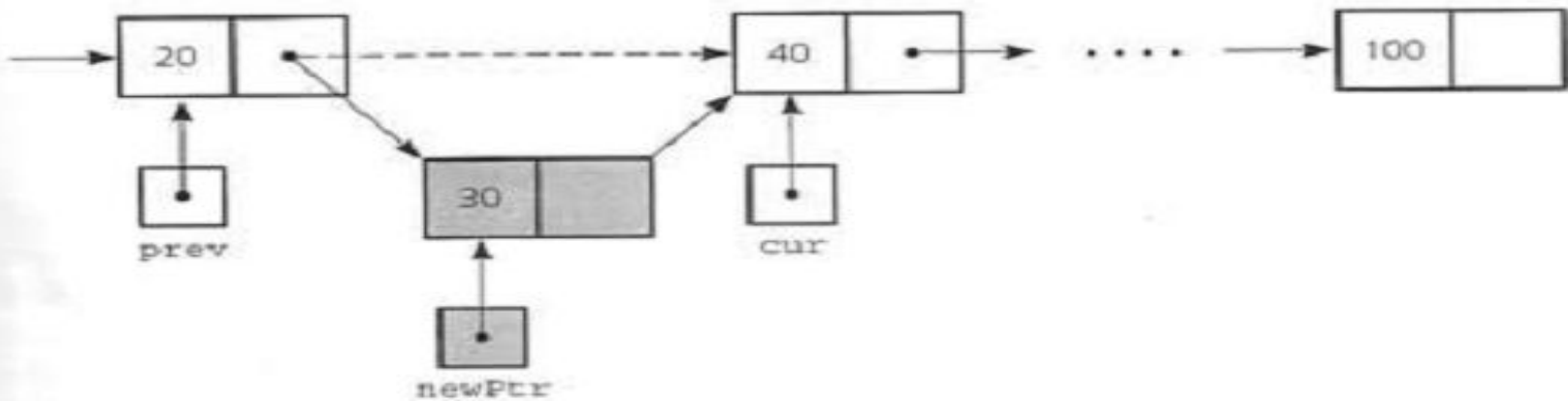


FIGURE 4-12

Inserting a new node into a linked list

```
newPtr->next = cur;  
prev->next = newPtr;
```

SPECIAL CASE: INSERTION AT START

- Inserting a node at start of the linked list:
`newPtr->next = head;`
`head = newPtr;`

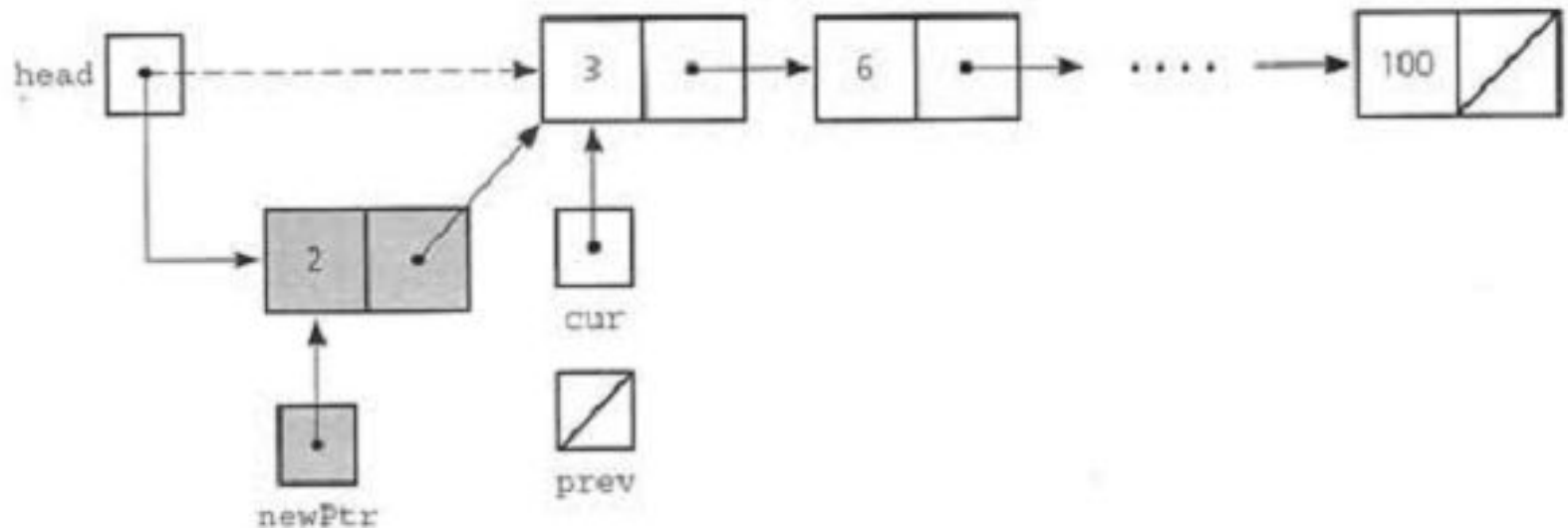


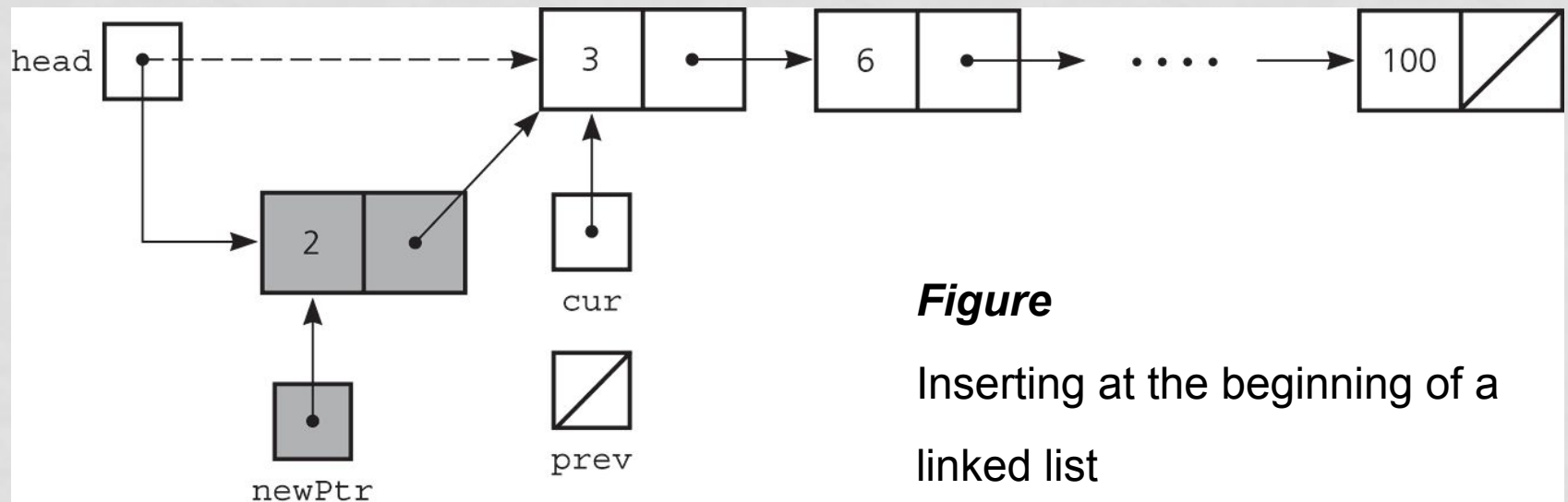
FIGURE 4-13

Inserting at the beginning of a linked list

INSERTING A NODE INTO A SPECIFIED POSITION OF A LINKED LIST

- To insert a node at the beginning of a linked list

```
newPtr->next = head;  
head = newPtr;
```



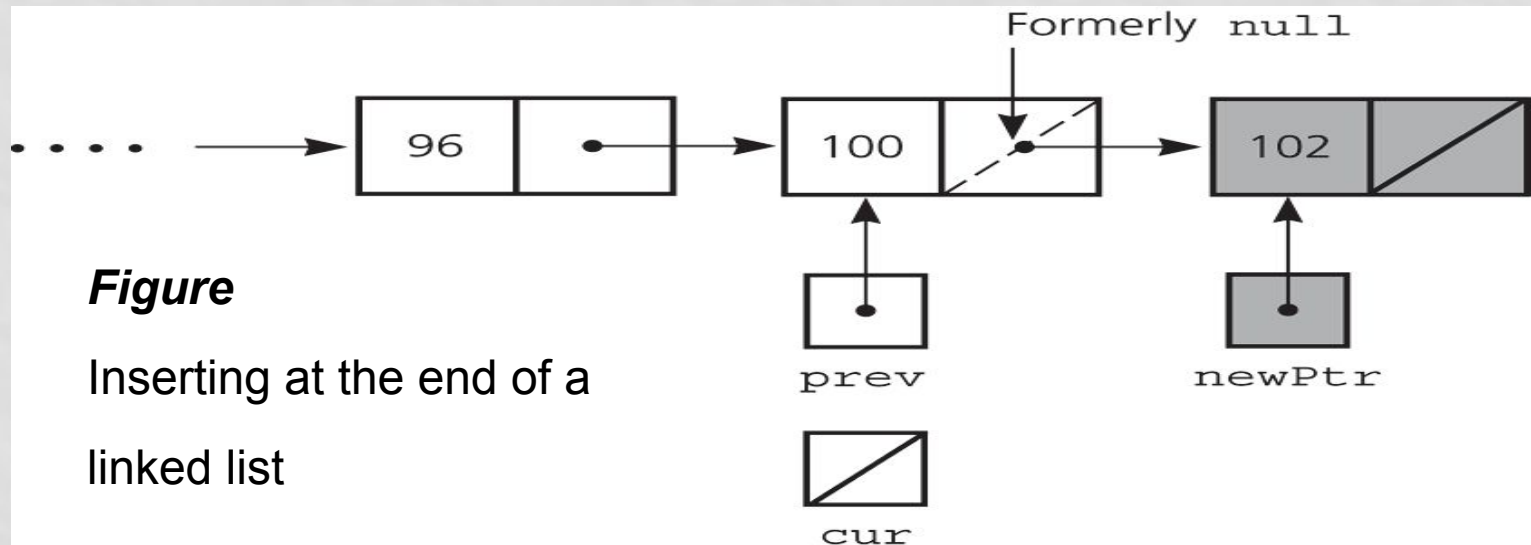
Figure

Inserting at the beginning of a linked list

INSERTING A NODE INTO A SPECIFIED POSITION OF A LINKED LIST

- Inserting at the end of a linked list is not a special case if *cur* is *NULL*

```
newPtr->next = cur;  
prev->next = newPtr;
```



SUMMARIZING INSERTION

- Three high-level steps
 1. Determine the point of insertion
 2. Create a new node and store the new item in it
 3. Connect the new node to the linked list by changing pointers

struct linkedlist	void fun()
<pre>{ linkedlist(); void insertAtStart(int c); void deleteAtStart(); void insertAtEnd(int c); void deleteAtEnd(); void traverse(); Bool isEmpty(); ~linkedlist(); }; void main() { linkedlist l; l.insertAtStart(9); linkedlist m=l; l.traverse(); }</pre>	<pre>{ linkedlist h; h.insertAtStart(9); h.insertAtStart(10); h.insertAtStart(1); h.insertAtStart(12); h.traverse(); }</pre>

Memory leakage?

DESTRUCTOR

- Use the destructor to release all the memory used by the list.
- Step through the list and delete each node one by one.

COPY CONSTRUCTOR

- Provide Copy Constructor to your ADT to ensure deep copying

```
struct linkedlist
```

```
{  
    linkedlist();  
    void insertAtStart(int c);  
    void deleteAtStart();  
    void deleteAtAnyPos();  
    void insertAtEnd(int c);  
    void deleteAtEnd();  
    void insertAtAnyPos();  
    void traverse();  
    Bool isEmpty();  
    ~linkedlist();  
    linkedlist(const linkedlist & h);  
};
```

ADVANTAGES

- Linked lists are a dynamic data structure, allocating the needed memory when the program is initiated.
- Insertion and deletion node operations are easily implemented in a linked list.
- Linear data structures such as stacks and queues are easily executed with a linked list.
- They can reduce access time and may expand in real time without memory overhead.

DISADVANTAGES

- Nodes in a linked list must be read in order from the beginning as linked lists are inherently sequential access.
- Nodes are stored in contiguously, greatly increasing the time required to access individual elements within the list.